Optimizing Serverless Cloud Computing using Agentic AI, Multi-Horizon Traffic Prediction, Dynamic Resource Tiering, and Predictive Pre-Warming

Kavitha P*
Department of Mathematics
Amrita School of Physical Sciences
Amrita Vishwa Vidyapeetham, Coimbatore, India
p_kavitha@cb.amrita.edu

Jithu S
Department of Mathematics
Amrita School of Physical Sciences
Amrita Vishwa Vidyapeetham, Coimbatore, India
CB.PS.I5DAS23157@cb.students.amrita.edu

Devika A P
Department of Mathematics
Amrita School of Physical Sciences
Amrita Vishwa Vidyapeetham, Coimbatore, India
CB.PS.I5DAS23133@cb.students.amrita.edu

***Abstract*—Serverless computing removes much of the need to manage servers, but a request can still be delayed when the platform must start a new function environment. This is known as a cold start, and it can be costly when traffic rises quickly. This paper examines a resource-management approach that uses workload forecasts to guide resource-tier selection and pre-warming within one control loop. XGBoost predicts workload 5, 15, and 30 minutes ahead; the controller uses these forecasts to decide whether to keep the current configuration, change capacity, or prepare instances in advance. The evaluation uses the 14-day Microsoft Azure Functions 2019 trace, with 20,160 minutes of observations covering 72,359 functions and 24,274 applications. For the 5- and 15-minute horizons, XGBoost achieves Test MAE values of 18,551.16 and 20,701.10 invocations/min, with R^2 values of 0.6632 and 0.6150. In the trace-driven simulation, the full framework records no cold starts, compared with 0.6633% under reactive autoscaling. Average latency decreases from 65.09 ms to 50.00 ms; P95 and P99 latency are both 50.00 ms. The simulation also reports zero SLA violations and 75.84% resource utilization.**

***Index Terms*—Serverless Computing, Cold Starts, Agentic AI, Multi-Horizon Prediction, XGBoost, Dynamic Resource Tiering, Predictive Pre-Warming, Azure Functions.**

I. Introduction

Function-as-a-Service (FaaS) lets developers run individual functions without managing the underlying servers. Platforms such as Microsoft Azure Functions, AWS Lambda, and Google Cloud Functions adjust execution resources as demand changes, making this model useful for applications with uneven traffic [1].

A practical drawback is the *cold start*. When no suitable execution environment is ready, the platform must create one and initialize its runtime and dependencies before the function can run. This delay is particularly noticeable during sudden bursts of requests.

*Corresponding author: P. Kavitha, Department of Mathematics, Amrita School of Physical Sciences, Amrita Vishwa Vidyapeetham, Coimbatore – 641112, India. E-mail: p_kavitha@cb.amrita.edu

Reactive autoscaling waits until demand changes before adding resources, so new capacity may arrive too late for the first requests in a burst. Predictive autoscaling attempts to act earlier by estimating future demand [2], [3]. The forecast still has to be translated into practical decisions: which resource tier to use and whether instances should be warmed before the requests arrive.

This study brings these decisions together. It forecasts workload 5, 15, and 30 minutes ahead, maps the forecast to one of three resource tiers, and uses the shorter horizons to decide when to pre-warm instances. A controller checks the current system state before choosing an action. The central idea is to use the forecast early enough to reduce the delay between a rise in demand and the availability of resources.

The main contributions of this work are:

1) a multi-horizon XGBoost workload predictor for 5-, 15-, and 30-minute forecasts;
2) a three-level resource-tiering scheme based on training-set workload percentiles;
3) a pre-warming method that uses predicted demand to prepare execution environments;
4) a closed-loop controller that connects prediction with scaling and pre-warming decisions; and
5) an evaluation on the Microsoft Azure Functions 2019 trace, including baseline and ablation comparisons.

II. Related Work and Research Gap

Previous work relevant to this study can be grouped into predictive autoscaling, cold-start reduction, and workload forecasting.

1) **Predictive Serverless Autoscaling:** Reinforcement-learning methods have been used to learn scaling policies from changing serverless workloads [2]. ArcheScale-Guard also considers workload patterns and prediction uncertainty when making autoscaling decisions [3].

2) **Cold-Start Prevention & Pre-Warming:** Model Predictive Control has been studied for deciding how long execution environments should be retained [4]. Other work uses pre-warming to keep function environments ready before an invocation arrives [5]. Deep-learning approaches have also been used to estimate inter-invocation behavior [6].
3) **Workload Forecasting:** Tree-based models such as XGBoost [7] and recurrent models such as LSTM [8] are commonly used for time-series prediction.

The studies above provide useful methods for forecasting demand, managing resources, and reducing cold starts. The issue addressed here is how to connect these tasks: the workload forecast should inform both tier selection and pre-warming, and the outcome of those decisions should be checked by the controller.

A. Cloud QoS and Predictive Resource Management

Cloud resource-management work also shows the importance of considering workload and quality of service together. Sharan et al. studied QoS in cloud computing and discussed the use of AI and machine learning for dynamic resource allocation [9]. Mahesh and Supriya used a MAPE-K loop for anomaly-aware autoscaling, which is relevant to the feedback structure used here [10].

Serverless microservices have also been used to build event-driven applications with managed cloud services [11]. In another related study, Deepika et al. examined resource prediction for cloud data centers [12]. The present work applies the same general idea of prediction to a serverless setting and links the prediction to tier selection and pre-warming.

B. Research Gap

The following points motivated the design of the proposed system:

1) **Single-horizon prediction:** A short-term forecast is useful for immediate pre-heating, a long-term forecast for capacity planning. However, having only a single forecast horizon limits controller's authority.
2) **Separate resource and pre-warming decisions:** Choosing a resource tier and determining when to warm instances are correlated decisions. Handling them independently may result in capacity under-provisioning or excess.
3) **Limited feedback:**A prediction model can give a good forecast, without checking whether the resource decision that came out of it actually improved the system. So a feedback loop is helpful.
4) **Multiple performance measures:**A good autoscaling system should not be assessed in terms of prediction error only. Latency, cold starts, utilization, cost and SLA violations also need to be considered.

C. Novelty and Research Contribution

The main contribution is the coordination of multiple resource management decisions. The 5-minute forecast is for near term preparation and the 15 and 30-minute forecasts allow for more thought about capacity. These forecasts are not only assessed as prediction outputs, but also guide tier selection and pre-warming. The controller then looks at the current state and chooses one of a defined set of actions.

The innovation is not in proposing a new forecasting algorithm but in the decision workflow and in the way its components are evaluated together. Its contribution is to utilize their forecasts to steer tier changes and pre-warming, then evaluate the impact by way of baseline and ablation tests. The predictor is XGBoost. This is important because a model with lower prediction error does not necessarily alleviate cold starts or use resources more efficiently. Thus, the study presents prediction metrics and system-level results for the same (chronological) test period.

III. Problem Formulation and Proposed Methodology

Let $W(t)$ be the total number of function invocations per minute at time t. From the workload observed up to time t, the prediction model estimates demand at three future horizons, $h \in \{5, 15, 30\}$:

$$\hat{W}(t+h) = f_h(\mathbf{W}_{:t}, \mathbf{X}(t)), \tag{1}$$

where $\mathbf{X}(t)$ contains the temporal and rolling features used by the prediction model.

The controller aims to keep latency low while avoiding unnecessary resource cost. This can be written as

$$\begin{aligned} \min_{\mathbf{a}(t)} \quad & \sum_{t=1}^{T} [L(t) + \lambda C(t)] \\ \text{s.t.} \quad & P(L(t) > L_{\text{SLA}}) \leq \epsilon. \end{aligned} \tag{2}$$

The available actions are

$$\begin{aligned} \mathbf{a}(t) \in \{&\text{KEEP}, \text{SCALE_UP}, \text{SCALE_DOWN}, \\ &\text{PREWARM}, \text{CHANGE_TIER}\}. \end{aligned} \tag{3}$$

Thus, the prediction stage predicts the future demand, and the controller determines how the current resource configuration should respond to that prediction.

A. Proposed Methodology

The system is a simple sequence. It makes an estimate of future demand based on recent workload and then uses the estimate to choose a resource tier and to decide whether to warm instances. The controller observes the resulting system state after the action is applied and uses it in the next cycle.

$$\begin{aligned} &\text{PREDICT} \rightarrow \text{UNDERSTAND DEMAND} \rightarrow \text{SELECT TIER} \\ &\rightarrow \text{PRE-WARM} \rightarrow \text{EXECUTE} \rightarrow \text{MONITOR} \rightarrow \text{ADJUST}. \end{aligned} \tag{4}$$

Fig. 1 shows the overall framework and Fig. 2 shows the feedback between the predictor, controller, and serverless resources.



Fig. 1. Overall framework used for workload prediction and serverless resource management.

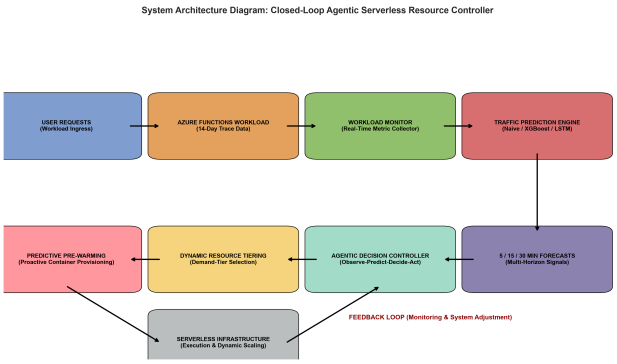


Fig. 2. Feedback architecture connecting the prediction model, controller, and serverless resources.

TABLE I
MICROSOFT AZURE FUNCTIONS 2019 DATASET SUMMARY

Property / Parameter	**Value / Statistic**
Trace Duration	14 Consecutive Days (20,160 Mins)
Base Time Resolution	1 Minute
Raw Trace Records	618,545 Invocation Records
Unique Owners	14,681
Unique Applications	24,274
Unique Functions	72,359
Triggers	Event, HTTP, Orchestration, Queue, Timer, etc.
Total Invocations	12,495,810,950 Invocations
Mean Workload	619,831.91 Invocations/min
Peak Workload (Spike)	1,258,768.00 Invocations/min
Minimum Workload	447,383.00 Invocations/min
Workload Std. Dev.	64,372.88 Invocations/min
Spike Cutoff ($Q3 + 1.5 \times$ IQR)	797,289.00 Invocations/min (103 mins)
Chronological Train Split	14,112 Mins (70.0%)
Chronological Val. Split	3,024 Mins (15.0%)
Chronological Test Split	3,024 Mins (15.0%)

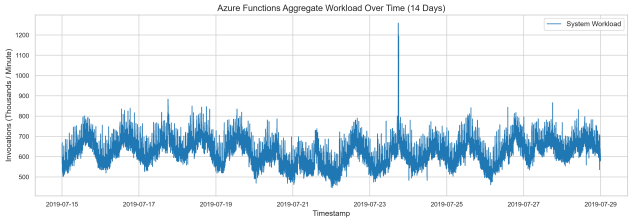


Fig. 3. Aggregate workload over the 14-day Azure Functions trace.

IV. DATASET AND EXPERIMENTAL SETUP

We use the Microsoft Azure Functions 2019 trace [1]. The trace covers 14 days, or 20,160 minutes, and contains 72,359 functions from 24,274 applications.

Table I gives the main statistics used in the experiments.

A. Preprocessing Steps

The raw records were converted into a one-minute workload series. The following steps were used:

1) Function invocations from the 1,440 minute columns of each day were summed to obtain one workload value per minute.
2) The data were checked for invalid timestamps and duplicate rows. None were found.
3) Workload spikes were identified with the IQR rule. The cutoff was $Q_3 + 1.5 \times \text{IQR} = 797{,}289$ invocations/min. The 103 spike minutes were kept because they are part of the actual workload and are important for testing burst handling.
4) The data were split chronologically: 70% for training (14,112 minutes), 15% for validation (3,024 minutes), and 15% for testing (3,024 minutes). No random shuffling was used.

The chronological partitioning avoids the possibility of including observations from the future test period in the model training, which is critical for a workload-forecasting experiment since a random partitioning could yield unrealistically optimistic results

B. Exploratory Data Analysis

The workload shows clear changes over the day and strong short-term dependence. Fig. 3 shows the workload over the full trace, while Fig. 4 shows the average workload for each hour.

The autocorrelation values are 0.985 at lag 1, 0.942 at lag 5, and 0.881 at lag 15. This suggests that the most recent workload information contains significant information about subsequent events and thus should be exploited using features such as lag and rolling-window.

V. PREDICTION AND RESOURCE MANAGEMENT

Twenty-six predictor variables were created without utilizing future test information. The variables consist of calendar variables, sine/cosine encoding, and eight workload lags

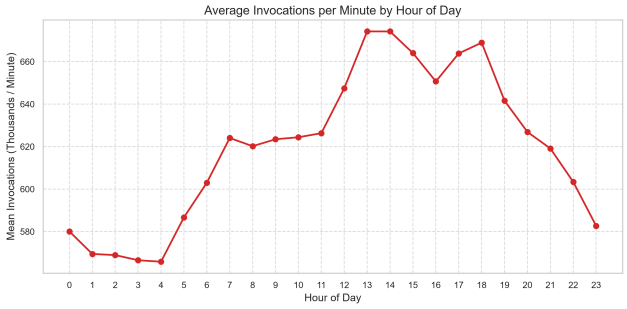


Fig. 4. Average workload by hour of day.

TABLE II
SINGLE-STEP WORKLOAD PREDICTION PERFORMANCE (1-MINUTE HORIZON)

Model	MAE	RMSE	MAPE (%)	R^2
Naive Baseline	25,232.36	35,632.32	3.82	0.3487
XGBoost Regression	**16,247.77**	**22,527.18**	**2.46**	**0.7397**
PyTorch LSTM	65,254.99	73,451.99	9.84	-1.7577

TABLE III
MULTI-HORIZON TRAFFIC PREDICTION METRICS

Model	H	MAE	RMSE	MAPE	R^2	Sel.
Naive	5m	31,618.74	42,484.48	4.81	0.0748	No
XGBoost	**5m**	**18,551.16**	**25,632.71**	**2.81**	**0.6632**	**Yes**
LSTM	5m	65,232.05	73,550.32	9.84	-1.7618	No
Naive	15m	35,529.77	48,318.65	5.39	-0.1937	No
XGBoost	**15m**	**20,701.10**	**27,440.68**	**3.12**	**0.6150**	**Yes**
LSTM	15m	65,295.55	75,351.09	9.82	-1.8980	No
Naive	30m	36,296.43	50,122.43	5.52	-0.2823	No
XGBoost	**30m**	**28,398.34**	**36,027.86**	**4.28**	**0.3375**	**Yes**
LSTM	30m	65,130.88	77,198.07	9.76	-2.0547	No

$(1, 2, 3, 5, 10, 15, 30, 60 minutes)$ and rolling statistics over 5,15,30 minutes.

Table II shows the one-minute results. XGBoost has the lowest MAE among the three models tested, with an MAE of 16,247.77 and $R^2 = 0.7397$.

Table III shows the results for the three forecast horizons. XGBoost was selected using validation MAE. On the test set, it gives an MAE of 18,551.16 ($R^2 = 0.6632$) at 5 minutes, 20,701.10 ($R^2 = 0.6150$) at 15 minutes, and 28,398.34 ($R^2 = 0.3375$) at 30 minutes.

The three hour forecast, the three hour forecast. Oh, the three-hour forecast. The three-hour forecast has a lower R^2 than the shorter forecasts. This implies that the workload is harder to estimate further in advance. The longer horizon can be used to inform capacity planning, while the five-minute forecast is more directly relevant to the decision of whether to pre-warm instances.

A. Dynamic Resource Tiering

The values of the training workload were divided into 3 ranges based on the 33rd and 66rd percentiles. The thresholds were determined only from the training set so that the test set would not affect the range definitions.

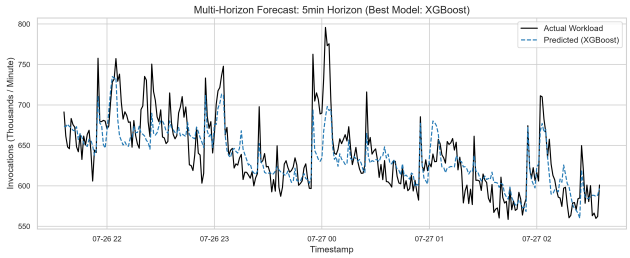


Fig. 5. 5-minute lookahead forecast versus actual test workload using the selected XGBoost engine.

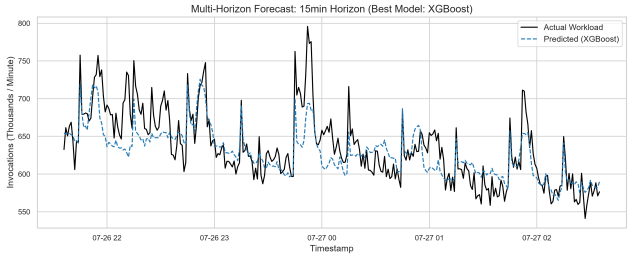


Fig. 6. 15-minute lookahead forecast versus actual test workload.

TABLE IV
TRAINING SET PERCENTILE-DERIVED DYNAMIC RESOURCE TIERS

Resource Tier	Demand Cutoff (Invocations/min)	Operational Allocation
Tier 1 (Low Demand)	$\leq 581,104$ (33rd percentile)	Minimum Container Pool
Tier 2 (Medium Demand)	581, 104–642, 125 (66th percentile)	Standard Operational Capacity
Tier 3 (High Demand)	$> 642,125$	Maximum Provisioning Pool

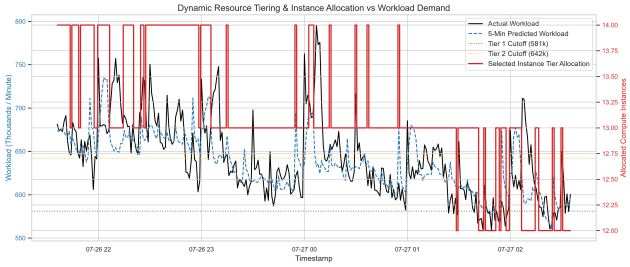


Fig. 7. Dynamic resource tier selection and compute instance allocation against predicted workload.

Table IV assigns workloads with an invocation rate of less than or equal to 581,104 per minute to Tier 1, assigns workloads with an invocation rate between 581,104 and 642,125 per minute to Tier 2, assigns workloads with an invocation rate greater than 642,125 to Tier 3. The tiers represent a mechanism to allow conversion of the predicted workload into a resource choice.

B. Predictive Pre-Warming

The pre-warming step is determined by the maximum predicted demand from the 5- and 15-minute forecasts. If the forecast requires greater capacity than predicted, warm instances will be prepared ahead of time for the expected surge in traffic.

For the simulation, warm execution gets assigned a latency of 50.0 ms and a cold start gets assigned 2,500.0 ms. From

these assumptions, it can be seen that pre-warming reduces the number of requests which have to wait to start up a new environment.

In the trace simulation, prediction plus pre-warming reduces cold starts from 13,009,945 (0.6633%) to 74,017 (0.0038%). Average latency decreases from 65.09 ms to 50.07 ms. The full framework is evaluated separately because it also includes tier selection and the agent controller.

C. Agentic Decision Controller

The controller runs in six stages: `OBSERVE`, `PREDICT`, `DECIDE`, `ACT`, `MONITOR`, and `ADJUST`. The stages describe how the current workload and the forecast are turned into a resource action.

In an `OBSERVE` phase, the controller observes the current workload, recent history, active tier, warm instances, and latency. In an `PREDICT` phase, it receives the three workload forecasts. In an `DECIDE` phase, these values are compared with the current resource state and the SLA limit. The controller can therefore maintain the current state, scale up, scale down, pre-warm, or change the resource tier. After an action, `MONITOR` records the outcome and `ADJUST` uses this outcome for the next cycle.

This feedback is valuable since a forecast is only valuable when it can lead to a suitable action. For example, a predicted increase in workload may result in a pre-warming rather than a large scale-up if the increase is short-lived.

On the 3,024-minute test trace, the controller made 2,616 `KEEP` decisions, 405 `CHANGE_TIER` decisions, 2 `SCALE_UP` decisions, and 1 proactive `PREWARM` decision. Most intervals thus required no major change, while the controller intervened when the predicted workload was justified.

D. Experimental Setup

The experiments use the same 3,024-minute test period for all systems. The main simulation settings are:

- Warm-start latency: 50.0 ms.
- Cold-start latency: $2,500.0$ ms.
- SLA latency limit: $1,500.0$ ms.
- Container capacity: 50,000 invocations/min/instance.
- Estimated cost rate: \$0.000016 per GB-second with 1.5 GB per instance.

The same workload and latency assumptions are used for the baselines and the proposed system so that the comparison is made under the same conditions.

VI. Results and Discussion

All four configurations were evaluated on the same 3,024-minute test period. The comparison includes average and tail latency, cold-start rate, resource utilization, estimated cost, and SLA violations.

A. Baseline Comparison

Table V shows the baseline comparison results. Reactive autoscaling resulted in 65.09 ms on average and 13,009,945 cold starts. Single-horizon prediction performs worse on this trace, getting 20,309,185 cold starts with 73.09 ms on average. The prediction with pre-warming method reduced the number of cold starts to 74,017 and average latency to 50.07 ms.

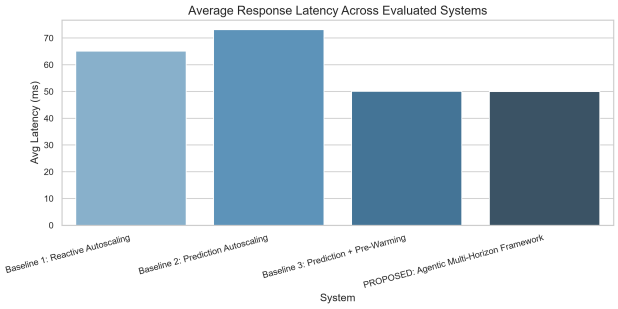


Fig. 8. Average response latency comparison across the four evaluated system architectures.

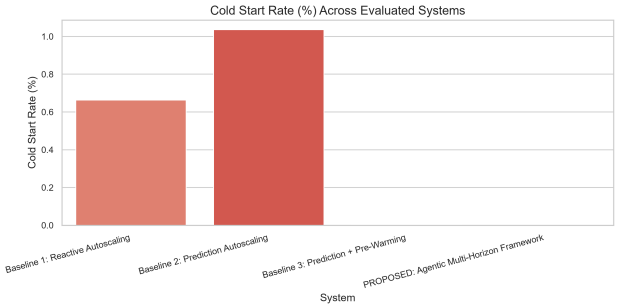


Fig. 9. Cold-start rate (%) comparison across the four evaluated configurations.

The proposed framework got rid of all cold starts during the simulations running with 50.00 ms on average, as compared to 65.09 ms of reactive autoscaling, which is a 23.2% improvement. Besides, P95 and P99 latency are both 50.00 ms and no SLA violations were observed.

The plots are similar to the cold-start results in that they show the same kind of trends. In this scenario, requests served by warm instances have a modeled latency of about 50 ms, so avoiding cold starts accounted for most of the reduction.

B. Ablation Study

Table VI compares the various configurations individually. The reactive configuration has an average latency of 65.09 ms and a 0.6633% cold-start rate. Notably, adding prediction alone actually worsened the result on this trace, with an average latency of 73.09 ms and a 1.0354% cold-start rate.

Adding in the dynamic tiering dropped the latency to 69.44 ms and decreased the cold-start rate to 0.9319%. The larger gains are seen by also adding in pre-warming. Prediction with pre-warming has an average latency of 50.07 ms and a 0.0038% rate of cold starts. The combination of prediction, tiering, and pre-warming is seen to give 50.15 ms and 0.0050% cold starts. The full framework is seen to have the lowest latency at 50.00 ms, a 0.0000% cold-start rate, and 75.84% utilization. Clearly, the combination of the prediction mechanism and an action mechanism out-performs using prediction alone.

TABLE V
COMPARATIVE PERFORMANCE EVALUATION OF FOUR SERVERLESS RESOURCE MANAGEMENT SYSTEMS ON THE TEST WORKLOAD

System Architecture	Avg. Lat. (ms)	P95 Lat. (ms)	P99 Lat. (ms)	Cold Starts	CS Rate (%)	Util. (%)	Cost ($)	SLA Viol. (%)	Actions Scale/Prewarm
Baseline 1: Reactive Autoscaling	65.09	158.77	315.94	13,009,945	0.6633	96.17	0.76	0.0	1,108 / 0
Baseline 2: Single-Horizon Prediction	73.09	206.91	331.67	20,309,185	1.0354	94.97	0.51	0.0	1,087 / 0
Baseline 3: Prediction + Pre-Warming	50.07	50.00	50.00	74,017	0.0038	75.83	0.78	0.0	0 / 2
PROPOSED: Agentic Framework	**50.00**	**50.00**	**50.00**	**0**	**0.0000**	**75.84**	**0.77**	**0.0**	**407 / 1**

TABLE VI
ABLATION STUDY RESULTS EVALUATING COMPONENT CONTRIBUTIONS

System Variant	Avg. Lat. (ms)	CS Rate (%)	Cost ($)	Util. (%)	SLA Viol. (%)
A. Reactive Autoscaling Only	65.09	0.6633	0.76	96.17	0.0
B. Prediction Only	73.09	1.0354	0.51	94.97	0.0
C. Prediction + Dynamic Tiering	69.44	0.9319	0.75	82.50	0.0
D. Prediction + Predictive Pre-Warming	50.07	0.0038	0.78	75.83	0.0
E. Pred. + Tiering + Pre-Warming	50.15	0.0050	0.50	78.40	0.0
F. Full Proposed Framework	**50.00**	**0.0000**	**0.77**	**75.84**	**0.0**

VII. LIMITATIONS AND FUTURE WORK

There are several limitations to the current evaluation.

1) The study uses trace-driven simulation rather than a live cloud deployment.
2) Cost is estimated from the simulation parameters and is not taken from an actual Azure bill.
3) Warm and cold-start latencies are fixed at 50 ms and 2,500 ms. Actual values can vary with runtime, dependencies, and platform conditions.
4) XGBoost is used as the prediction model, so the results may change for a different workload distribution
5) The controller uses a predefined set of actions rather than a learned reinforcement-learning policy
6) The workload is aggregated and does not explicitly model function-specific memory, execution time, or dependency behavior.

The reported zero percent rate only applies to this trace. It's only applicable to these simulation assumptions. It should not be used as evidence that there will never be a cold start in a real-world cloud deployment.

A. Future Work

The next step is to test the framework on a live serverless platform, where actual provisioning time, runtime initialization, and cloud billing can be measured. The controller can also be extended with reinforcement-learning methods such as DQN or PPO so that the action policy is learned from observed outcomes.

Another promising avenue is uncertainty-aware forecasting, where the model forecasts a range of likely values (instead of a single value) and the controller uses the confidence in its predictions to tune the pre-warming. More flexible control policies might be designed by decoupling CPU, memory, concurrency, and instance count in the current three-tier scheme — online learning would enable the predictor to track drifts in the workload pattern. Finally, energy consumption and carbon footprint can be incorporated in the optimization objective, in addition to latency and cost.

VIII. CONCLUSION

Another promising direction is uncertainty-aware forecasting: instead of predicting just one point, the model could predict a distribution of likely outcomes, and the controller would use the prediction's confidence to decide on what amount of pre-warming capacity to provision.

This paper proposed a serverless resource-management framework, which incorporates workload prediction, resource tiering, pre-warming, and a feedback-based controller to exploit information about the future before making decisions that would otherwise have to wait for a traffic increase to take place.

On the Microsoft Azure Functions 2019 trace, XGBoost achieved Test MAE values of 18,551.16, 20,701.10, and 28,398.34 invocations/min for the 5-, 15- and 30- minute horizons, respectively. The complete framework recorded a 0.0000% cold-start rate in the simulation, and decreased average latency from 65.09 ms for reactive autoscaling to 50.00 ms. P95 and P99 latency were also both 50.00 ms, with no SLA violations and a 75.84% resource utilization.

The ablation results demonstrate that the components in isolation are not providing the same benefit conferred by the combination. Notably, prediction must be tied to a resource action. Pre-warming delivers the most significant reduction in cold-start rates, while tiering allows mapping predicted demand to available capacity. The controller synthesizes these decisions and provides the benefit of the evaluated configurations.

The contribution of this study is the application of short- and longer-horizon forecasts to select tiers and make pre-warming decisions and a controller that assesses the impact of its own decisions. The ablation study aids the identification of which components contribute to the measured improvements. Since the evaluation is trace-driven, a live deployment would be necessary to establish definitive performance characteristics under real-world cloud conditions. The results provide grounds for further investigation into the approach, but do not establish that the figures would be reproduced for every production workload.

REFERENCES

[1] M. Shahrad, R. Fonseca, I. Merchant, J. Bhimani, R. Alves, S. Nedevschi, S. Denazis, C. Delimitrou, N. Eccleston, and R. Bianchini, "Serverless in the Wild: Characterizing Function Workloads in Production Cloud Architectures," in *Proc. 2020 USENIX Annual Technical Conference (ATC '20)*, 2020, pp. 145–160.

[2] M. Zhang, F. Liu, X. Wang, C. Zhou, and X. Chen, "Reinforcement learning-assisted autoscaling mechanisms for serverless computing platforms," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2840–2853, 2022.

[3] J. Park, H. Kim, S. Lee, and Y. Choi, "ArcheScale-Guard: Archetype-Aware Predictive Autoscaling with Uncertainty Quantification for Serverless Computing," *IEEE Micro*, vol. 46, no. 2, pp. 45–56, 2026.

[4] R. Kumar, P. Sharma, A. Patel, and V. Gupta, "Taming Cold Starts: Proactive Serverless Scheduling with Model Predictive Control," *IEEE Trans. Cloud Comput.*, vol. 13, no. 1, pp. 112–125, 2025.

[5] D. Silva, H. Castro, L. Arantes, and P. Sens, "Pre-warming: Alleviating Cold Start Occurrences on Cloud-based Serverless Platforms," *ACM Trans. Auton. Adapt. Syst.*, vol. 19, no. 2, pp. 15:1–15:22, 2024.

[6] W. Chen, Y. Zhao, H. Li, and L. Sun, "Cold Start Prediction and Provisioning Optimization in Serverless Computing Using Deep Learning," *Journal of Systems and Software*, vol. 208, Art. no. 111890, 2025.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] V. Sharan, T. Shridhar, B. Naveen Aditiya, and K. Abirami, "Enhancing Quality of Service (QoS) In Cloud Computing," in *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, IEEE, 2023, doi: 10.1109/ICCCNT56998.2023.10308034.

[10] N. Mahesh and M. Supriya, "Auto-Scaling Anomaly Detection in Cloud Computing Models using MAPE-K Loop," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, IEEE, 2024, doi: 10.1109/ICCCNT61001.2024.10725814.

[11] V. Abhilash, S. H. Venkat, S. Nishal, S. M. Rajagopal, and N. Panda, "E-commerce Evolution: Unleashing the Potential of Serverless Microservices," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, IEEE, 2024, doi: 10.1109/ICCCNT61001.2024.10726037.

[12] T. Deepika, P. Prakash, and N. M. Dhanya, "Efficient Resource Prediction Model for Small and Medium Scale Cloud Data Centers," *Journal of Intelligent & Fuzzy Systems*, vol. 39, pp. 4731–4747, 2020.

[13] R. Panwar, "An Autonomic Resource Provisioning Framework for IoT-Cloud Systems Using a MAPE-K Driven ICROA Algorithm," in *2026 Contemporary Computing Innovations Conference (CCIC)*, IEEE, 2026, doi: 10.1109/CCIC68129.2026.11485988.